

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

M NAIMI SAI GANESH(1WA24CS166)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **M NAIMI SAI GANESH(1WA24CS166)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Ramya K.M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph. -	4-7
2	Implement Johnson Trotter algorithm to generate permutations.	8-9
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	10-12
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	13-14
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	15-16
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	17-20
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	21-25
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	26-29
9	Implement Fractional Knapsack using Greedy technique.	30-31
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	32-33
11	Implement "N-Queens Problem" using Backtracking.	34-36

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1: Topological Sorting

```
#include <stdio.h>

int main()
{
    int V;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    int g[V][V], indegree[V];

    printf("Enter adjacency matrix:\n");

    for(int i = 0; i < V; i++)
    {
        indegree[i] = 0;

        for(int j = 0; j < V; j++)
            scanf("%d", &g[i][j]);
    }

    // Calculate indegree
    for(int i = 0; i < V; i++)
        for(int j = 0; j < V; j++)
            if(g[i][j])
                indegree[j]++;

    printf("Topological Order: ");

    for(int k = 0; k < V; k++)
    {
        for(int i = 0; i < V; i++)
        {
            if(indegree[i] == 0)
            {
                printf("%d ", i);

                indegree[i] = -1;

                for(int j = 0; j < V; j++)
                    if(g[i][j])
                        indegree[j]--;
            }
        }
    }
```

```

    }

    return 0;
}

```

Output:

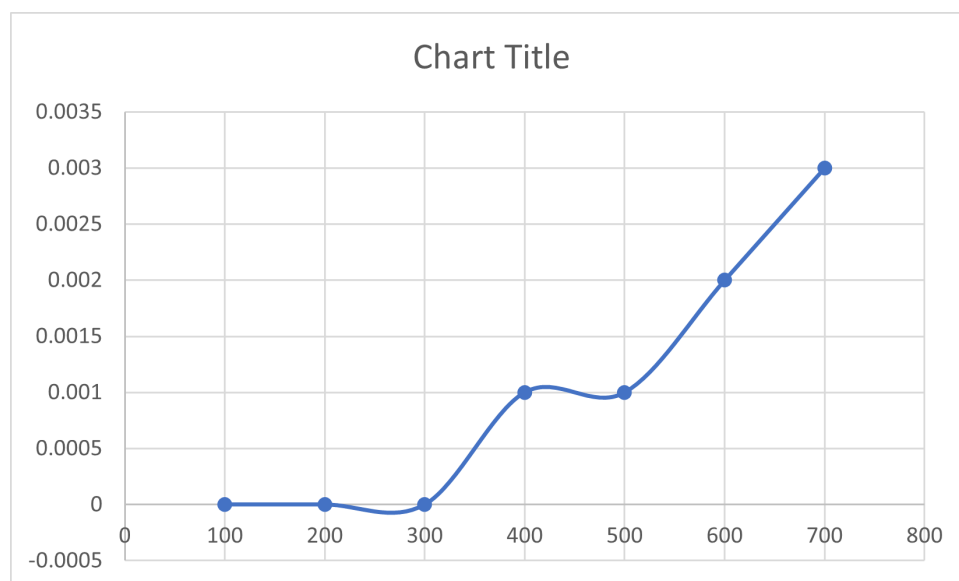
```

Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab" && gcc tempCodeRunnerFile.c -o tempCodeRunnerFile && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\tempCodeRunnerFile"
Enter number of vertices: 6
Enter adjacency matrix:
0 1 1 0 0 0
0 0 0 1 0 0
0 0 0 1 1 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0
Topological Order: 0 1 2 3 4 5
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>

```

Graph for Large inputs:



LeetCode Exercise on Topological Sorting:

Problem 2127

```
int maximumInvitations(int* fav, int n) {  
    int indeg[n], dist[n];  
    memset(indeg, 0, sizeof(indeg));  
    for (int i = 0; i < n; i++) indeg[fav[i]]++;  
    int q[n], f = 0, r = 0;  
    for (int i = 0; i < n; i++) {  
        dist[i] = 1;  
        if (!indeg[i]) q[r++] = i;  
    }  
    while (f < r) {  
        int u = q[f++], v = fav[u];  
        if (dist[v] < dist[u] + 1) dist[v] = dist[u] + 1;  
        if (--indeg[v] == 0) q[r++] = v;  
    }  
    int maxCycle = 0, pairSum = 0;  
    char vis[n];  
    memset(vis, 0, sizeof(vis));  
    for (int i = 0; i < n; i++) {  
        if (indeg[i] && !vis[i]) {  
            int len = 0, cur = i;  
            while (!vis[cur]) {  
                vis[cur] = 1;  
                cur = fav[cur];  
                len++;  
            }  
            if (len == 2) {  
                pairSum += len;  
            }  
        }  
    }  
    return pairSum;  
}
```

```

        int a = i, b = fav[i];

        pairSum += dist[a] + dist[b];

    } else if (len > maxCycle) {

        maxCycle = len;

    }

}

}

return maxCycle > pairSum ? maxCycle : pairSum;

}

```

Output:

The screenshot shows the LeetCode interface for problem 2127. The problem description states: "A company is organizing a meeting and has a list of n employees, waiting to be invited. They have arranged for a large circular table, capable of seating any number of employees. The employees are numbered from 0 to $n - 1$. Each employee has a **favorite** person and they will attend the meeting **only if** they can sit next to their favorite person at the table. The favorite person of an employee is **not** themselves. Given a 0-indexed integer array `favorite`, where `favorite[i]` denotes the favorite person of the i^{th} employee, return the **maximum number of employees** that can be invited to the meeting." Example 1 shows a circular table with two employees, 1 and 2, seated next to each other.

The test result section shows "Accepted" with a runtime of 0 ms. The input is `favorite = [2, 2, 1, 2]` and the output is `3`. The expected output is also `3`.

Lab program 2: Johnson Trotters Algorithm for Generating Permutations

```
#include <stdio.h>
```

```
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}
```

```
void print(int a[], int n)
{
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}
```

```
int mobile(int a[], int dir[], int n)
{
    int m = 0, pos = -1;

    for(int i = 0; i < n; i++)
    {
        if(dir[i] == -1 && i > 0 &&
            a[i] > a[i-1] && a[i] > m)
        {
            m = a[i];
            pos = i;
        }

        if(dir[i] == 1 && i < n-1 &&
            a[i] > a[i+1] && a[i] > m)
        {
            m = a[i];
            pos = i;
        }
    }

    return pos;
}
```

```
int main()
{
    int n;
```



```

printf("Enter n: ");
scanf("%d", &n);

int a[n], dir[n];

for(int i = 0; i < n; i++)
{
    a[i] = i + 1;
    dir[i] = -1;
}

print(a, n);

while(1)
{
    int pos = mobile(a, dir, n);

    if(pos == -1)
        break;

    int next = pos + dir[pos];

    swap(&a[pos], &a[next]);
    swap(&dir[pos], &dir[next]);

    pos = next;

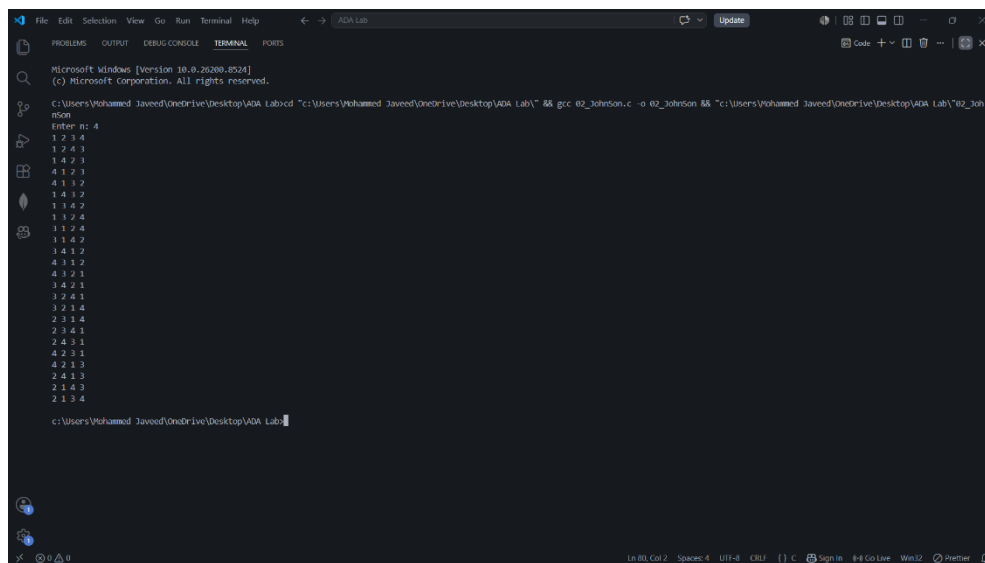
    for(int i = 0; i < n; i++)
        if(a[i] > a[pos])
            dir[i] *= -1;

    print(a, n);
}

return 0;
}

```

Output:



```
Microsoft Windows [Version 10.0.26280.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mohamed Javed\OneDrive\Desktop\ADA Lab> cd "c:\Users\Mohamed Javed\OneDrive\Desktop\ADA Lab\" && gcc 02_johnson.c -o 02_johnson && "c:\Users\Mohamed Javed\OneDrive\Desktop\ADA Lab\02_johnson"
Enter n: 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 1 2
1 4 3 2
1 3 4 2
1 1 2 4
1 1 2 4
1 1 4 2
1 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
1 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4

c:\Users\Mohamed Javed\OneDrive\Desktop\ADA Lab>
```

Lab program 3: Merge Sort

```
#include <stdio.h>
```

```
#include <time.h>
```

```
void merge(int a[], int l, int m, int r)
```

```
{
    int i = l, j = m + 1, k = 0, temp[1000];
```

```
    while(i <= m && j <= r)
```

```
    {
        if(a[i] < a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }
```

```
    while(i <= m)
        temp[k++] = a[i++];
```

```
    while(j <= r)
        temp[k++] = a[j++];
```

```
    for(i = l, k = 0; i <= r; i++, k++)
        a[i] = temp[k];
}
```

```
void mergeSort(int a[], int l, int r)
```

```
{
```

```

    if(l < r)
    {
        int m = (l + r) / 2;

        mergeSort(a, l, m);
        mergeSort(a, m + 1, r);

        merge(a, l, m, r);
    }
}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int a[n];

    printf("Enter elements:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    clock_t start = clock();

    mergeSort(a, 0, n - 1);

    clock_t end = clock();

    printf("Sorted array:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

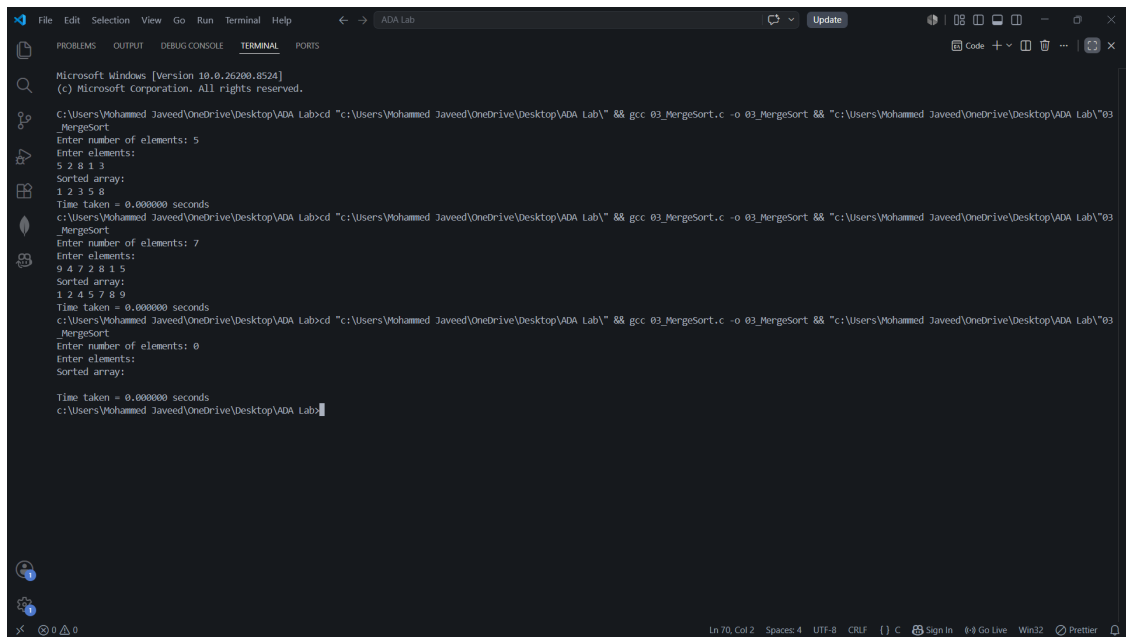
    double time_taken =
    (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nTime taken = %f seconds", time_taken);

    return 0;
}

```

Output:

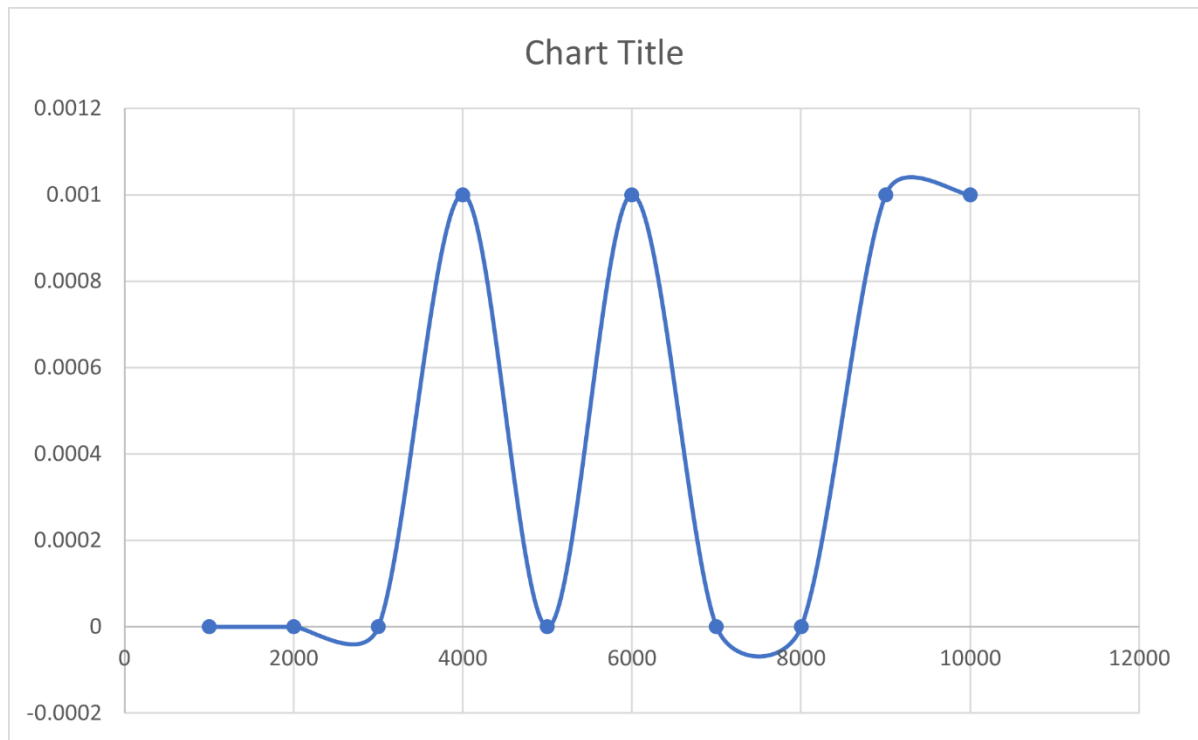


```
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 03_MergeSort.c -o 03_MergeSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\03_MergeSort"
Enter number of elements: 5
Enter elements:
5 2 8 1 3
Sorted array:
1 2 3 5 8
Time taken = 0.000000 seconds
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 03_MergeSort.c -o 03_MergeSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\03_MergeSort"
Enter number of elements: 7
Enter elements:
9 4 7 2 8 1 5
Sorted array:
1 2 4 5 7 8 9
Time taken = 0.000000 seconds
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 03_MergeSort.c -o 03_MergeSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\03_MergeSort"
Enter number of elements: 0
Enter elements:
Sorted array:

Time taken = 0.000000 seconds
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>
```

Graph for Large Inputs:



Lab program 4: Quick Sort

```
#include <stdio.h>
#include <time.h>

void quick(int a[], int l, int r)
{
    if(l >= r) return;

    int i = l, j = r, p = a[l];

    while(i < j)
    {
        while(a[i] <= p && i < r) i++;
        while(a[j] > p) j--;

        if(i < j)
        {
            int t = a[i];
            a[i] = a[j];
            a[j] = t;
        }
    }

    a[l] = a[j];
    a[j] = p;

    quick(a, l, j-1);
    quick(a, j+1, r);
}

int main()
{
    int n;

    printf("Enter n: ");
    scanf("%d",&n);

    int a[n];

    for(int i=0;i<n;i++)
        scanf("%d",&a[i]);

    clock_t s = clock();
```

```

quick(a,0,n-1);

clock_t e = clock();

printf("Sorted array:\n");

for(int i=0;i<n;i++)
    printf("%d ",a[i]);

printf("\nTime=%f",
(double)(e-s)/CLOCKS_PER_SEC);
}

```

Output:

```

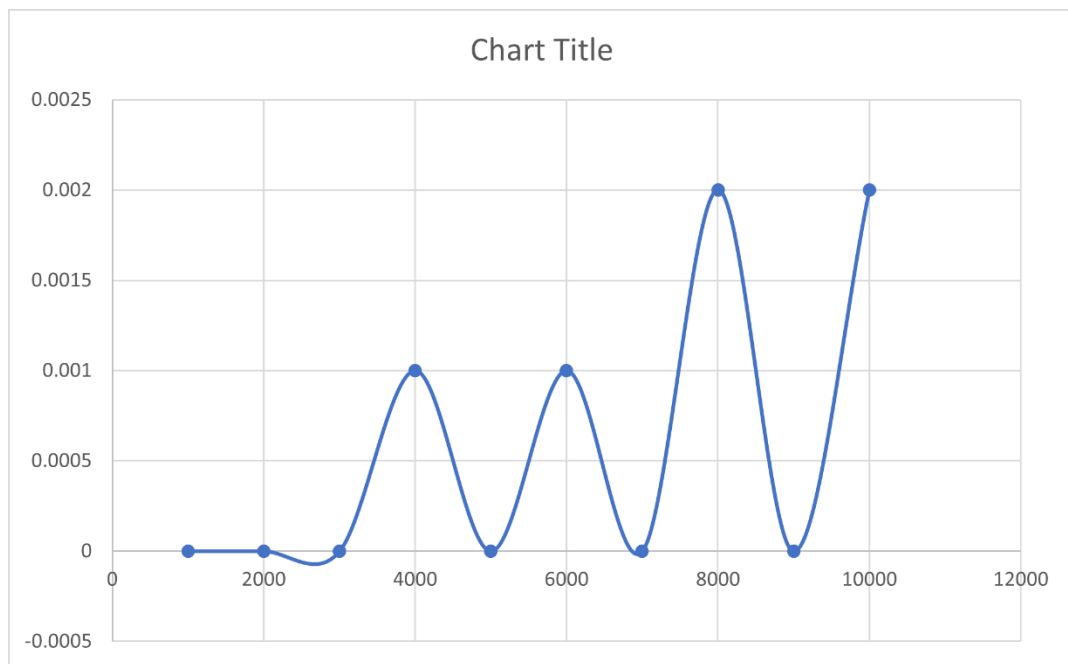
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 04_QuickSort.c -o 04_QuickSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\04_QuickSort"
Enter n: 5
5 2 8 1 3
Sorted array:
1 2 3 5 8
Time=0.000000
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 04_QuickSort.c -o 04_QuickSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\04_QuickSort"
Enter n: 7
9 4 7 2 8 1 5
Sorted array:
1 2 4 5 7 8 9
Time=0.000000
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 04_QuickSort.c -o 04_QuickSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\04_QuickSort"
Enter n: 0
Sorted array:

Time=0.000000
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>

```

Graph for Large Inputs:



Lab program 5: Heap Sort

```
#include <stdio.h>
void heapify(int a[], int n, int i)
{
    int largest=i,l=2*i+1,r=2*i+2;

    if(l<n && a[l]>a[largest])
        largest=l;

    if(r<n && a[r]>a[largest])
        largest=r;

    if(largest!=i)
    {
        int t=a[i];
        a[i]=a[largest];
        a[largest]=t;

        heapify(a,n,largest);
    }
}
```

```
int main()
{
    int n;
    printf("Enter n: ");
    scanf("%d",&n);

    int a[n];

    for(int i=0;i<n;i++)
        scanf("%d",&a[i]);

    for(int i=n/2-1;i>=0;i--)
        heapify(a,n,i);

    for(int i=n-1;i>0;i--)
    {
        int t=a[0];
        a[0]=a[i];
        a[i]=t;

        heapify(a,i,0);
    }
    printf("Sorted array:\n");
}
```

```

for(int i=0;i<n;i++)
    printf("%d ",a[i]);
}

```

Output:

```

Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 05_HeapSort.c -o 05_HeapSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\05_HeapSort
Enter n: 5
5 2 8 1 3
Sorted array:
1 2 3 5 8

c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 05_HeapSort.c -o 05_HeapSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\05_HeapSort
Enter n: 7
9 4 7 2 8 1 5
Sorted array:
1 2 4 5 7 8 9

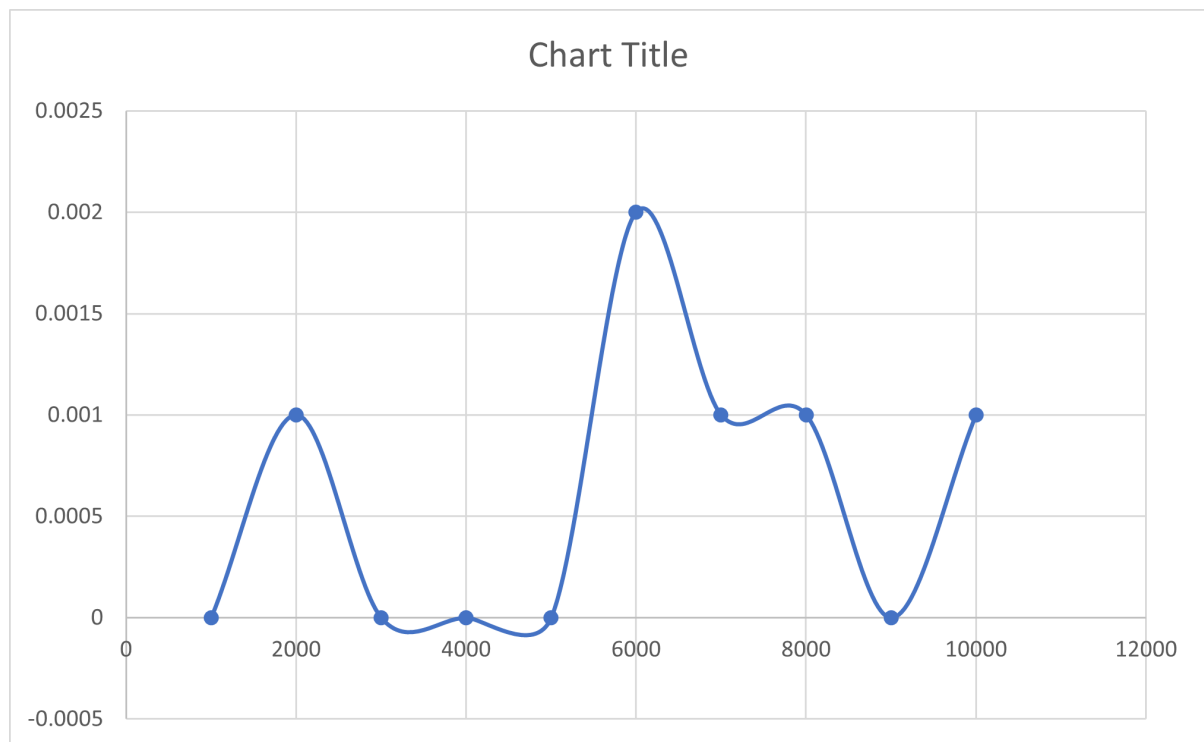
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 05_HeapSort.c -o 05_HeapSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\05_HeapSort
Enter n: 0
Sorted array:

c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 05_HeapSort.c -o 05_HeapSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\05_HeapSort
Enter n: kdjs

c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 05_HeapSort.c -o 05_HeapSort && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\05_HeapSort
Enter n: 4
1 2 3 4
Sorted array:
1 2 3 4
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>

```

Graph for Large Inputs:



Lab program 6: 0/1 KnapSack

```
#include <stdio.h>
```

```
int max(int a,int b)
{
    return a>b?a:b;
}
```

```
int main()
{
    int n,W;

    printf("Enter items and capacity: ");
    scanf("%d%d",&n,&W);

    int wt[n],val[n];

    for(int i=0;i<n;i++)
        scanf("%d",&wt[i]);

    for(int i=0;i<n;i++)
        scanf("%d",&val[i]);
```

```

int dp[n+1][W+1];

for(int i=0;i<=n;i++)
{
    for(int w=0;w<=W;w++)
    {
        if(i==0 || w==0)
            dp[i][w]=0;

        else if(wt[i-1]<=w)
            dp[i][w]=max(val[i-1]+dp[i-1][w-wt[i-1]],
                        dp[i-1][w]);

        else
            dp[i][w]=dp[i-1][w];
    }
}

printf("Maximum Profit=%d",dp[n][W]);
}

```

Output:

```

Microsoft Windows [Version 10.0.26300.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 06_01Knapsack.c -o 06_01Knapsack && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\"
06_01Knapsack
Enter items and capacity: 3 50
10 20 30
60 100 120
Maximum Profit=220
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 06_01Knapsack.c -o 06_01Knapsack && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\"
06_01Knapsack
Enter items and capacity: 4 7
1 3 4 5
1 4 5 7
Maximum Profit=9
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>

```

LeetCode 1626

```
int cmp(const void *a, const void *b) {  
    int *x = (int *)a, *y = (int *)b;  
    return x[0] == y[0] ? x[1] - y[1] : x[0] - y[0];  
}  
  
int bestTeamScore(int* scores, int scoresSize, int* ages, int agesSize) {  
    int p[scoresSize][2];  
  
    for (int i = 0; i < scoresSize; i++) {  
        p[i][0] = ages[i];  
        p[i][1] = scores[i];  
    }  
  
    qsort(p, scoresSize, sizeof(p[0]), cmp);  
  
    int dp[scoresSize], ans = 0;  
    for (int i = 0; i < scoresSize; i++) {  
        dp[i] = p[i][1];  
        for (int j = 0; j < i; j++)  
            if (p[j][0] <= p[i][0])  
                dp[i] = dp[i] > dp[j] + p[i][1]  
                    ? dp[j] + p[i][1]  
                    : dp[j] + p[i][1];  
        if (dp[i] > ans) ans = dp[i];  
    }  
    return ans;  
}
```

Output:

The screenshot shows the LeetCode interface for the problem "1626. Best Team With No Conflicts". The problem is categorized as "Medium". The description states: "You are the manager of a basketball team. For the upcoming tournament, you want to choose the team with the highest overall score. The score of the team is the **sum** of scores of all the players in the team. However, the basketball team is not allowed to have **conflicts**. A **conflict** exists if a younger player has a **strictly higher** score than an older player. A conflict does **not** occur between players of the same age. Given two lists, `scores` and `ages`, where each `scores[i]` and `ages[i]` represents the score and age of the i^{th} player, respectively, return the **highest overall score of all possible basketball teams**."

Example 1:
Input: `scores = [1,3,5,10,15]`, `ages = [1,2,3,4,5]`
Output: 34
Explanation: You can choose all the players.

The right side of the interface shows the "Test Result" section, which is "Accepted" with a runtime of 0 ms. It displays three test cases, all of which are passed. The input for Case 1 is `scores = [1,3,5,10,15]` and `ages = [1,2,3,4,5]`. The output is 34, which matches the expected result.

Lab program 7: Floyd's Algorithm

```
#include <stdio.h>

#define INF 999

int main()
{
    int n;

    printf("Enter vertices: ");
    scanf("%d",&n);

    int a[n][n];

    printf("Enter matrix:\n");

    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            scanf("%d",&a[i][j]);

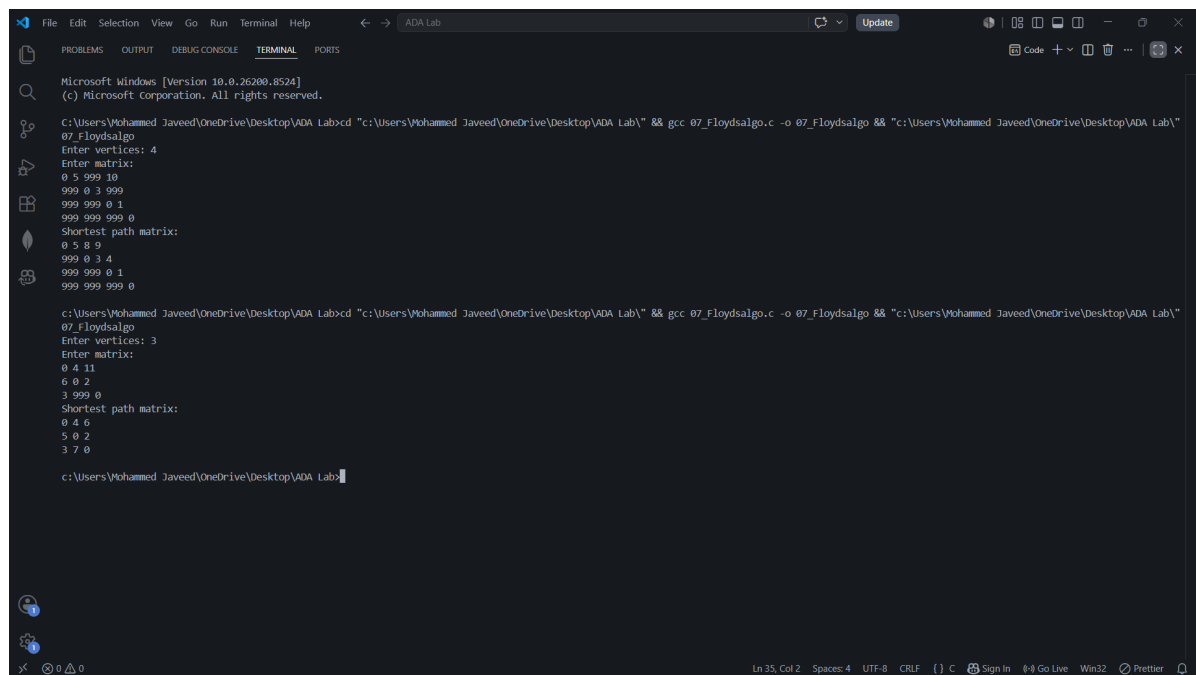
    for(int k=0;k<n;k++)
        for(int i=0;i<n;i++)
            for(int j=0;j<n;j++)
                if(a[i][k]+a[k][j]<a[i][j])
                    a[i][j]=a[i][k]+a[k][j];

    printf("Shortest path matrix:\n");

    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
            printf("%d ",a[i][j]);

        printf("\n");
    }
}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 07_Floydsalgo.c -o 07_Floydsalgo && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\"
07_Floydsalgo
Enter vertices: 4
Enter matrix:
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0
Shortest path matrix:
0 5 8 9
999 0 3 4
999 999 0 1
999 999 999 0

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 07_Floydsalgo.c -o 07_Floydsalgo && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\"
07_Floydsalgo
Enter vertices: 3
Enter matrix:
0 4 11
0 0 2
3 999 0
Shortest path matrix:
0 4 6
5 0 2
3 7 0

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>
```


LeetCode 1976

```
long long MOD = 1000000007LL;

int countPaths(int n, int** roads, int roadsSize, int* roadsColSize) {

    long long dist[201][201];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            dist[i][j] = (i == j) ? 0 : 1e18;
    }

    for (int i = 0; i < roadsSize; i++) {
        int u = roads[i][0];
        int v = roads[i][1];
        int w = roads[i][2];
        dist[u][v] = w;
        dist[v][u] = w;
    }

    // Floyd-Warshall
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];

    long long ways[201] = {0};
    ways[0] = 1;
    int used[201] = {0};
```

```

for (int step = 0; step < n; step++) {

    int u = -1;

    for (int i = 0; i < n; i++)

        if (!used[i] &&

            (u == -1 || dist[0][i] < dist[0][u]))

            u = i;

    used[u] = 1;

    for (int r = 0; r < roadsSize; r++) {

        int a = roads[r][0];

        int b = roads[r][1];

        int w = roads[r][2];

        if (u == a &&

            dist[0][a] + w == dist[0][b]) {

            ways[b] = (ways[b] + ways[a]) % MOD;

        }

        if (u == b && dist[0][b] + w == dist[0][a]) {

            ways[a] = (ways[a] + ways[b]) % MOD;

        }

    }

}

return (int)ways[n - 1];

}

```

Output:

The screenshot shows the LeetCode interface for the problem "1776. Number of Ways to Arrive at Destination". The problem is marked as "Solved" and "Medium" difficulty. The description states: "You are in a city that consists of n intersections numbered from 0 to $n - 1$ with bi-directional roads between some intersections. The inputs are generated such that you can reach any intersection from any other intersection and that there is at most one road between any two intersections. You are given an integer n and a 2D integer array `roads` where `roads[i] = [ui, vi, timei]` means that there is a road between intersections u_i and v_i that takes $time_i$ minutes to travel. You want to know in how many ways you can travel from intersection 0 to intersection $n - 1$ in the shortest amount of time. Return the number of ways you can arrive at your destination in the shortest amount of time. Since the answer may be large, return it modulo $10^9 + 7$."

Example 1:

The diagram shows a single intersection labeled 0.

The test results section shows "Accepted" with a runtime of 0 ms. The input is `n = 7` and `roads = [[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]]`. The output is `4` and the expected result is `4`.

Lab program 8(a): Prim's Algorithm

```
#include <stdio.h>

#define V 5
#define INF 999

int main()
{
    int g[V][V]={
        {0,2,0,6,0},
        {2,0,3,8,5},
        {0,3,0,0,7},
        {6,8,0,0,9},
        {0,5,7,9,0}
    };

    int selected[V]={0};

    selected[0]=1;

    int edges=0,cost=0;

    while(edges<V-1)
    {
        int min=INF,x=0,y=0;

        for(int i=0;i<V;i++)
        {
            if(selected[i])
            {
                for(int j=0;j<V;j++)
                {
                    if(!selected[j] && g[i][j])
                    {
                        if(g[i][j]<min)
                        {
                            min=g[i][j];
                            x=i;
                            y=j;
                        }
                    }
                }
            }
        }
    }
}
```

```

printf("%d-%d=%d\n",x,y,min);

cost+=min;
selected[y]=1;
edges++;
}

printf("Cost=%d",cost);
}

```

Output:



```

Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 08_PrimsAlgo.c -o 08_PrimsAlgo && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\08_PrimsAlgo"
0-1-2
1-2-3
1-4-5
0-3-6
Cost=16
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>

```

Lab program 8(b): Kruskal's Algorithm

```
#include <stdio.h>

int parent[10];

int find(int i)
{
    while(parent[i])
        i=parent[i];

    return i;
}

void uni(int i,int j)
{
    parent[j]=i;
}

int main()
{
    int n=4,min,cost=0;

    int a[4][4]={
        {0,10,6,5},
        {10,0,0,15},
        {6,0,0,4},
        {5,15,4,0}
    };

    int edges=0;

    while(edges<n-1)
    {
        min=999;

        int x=0,y=0;

        for(int i=0;i<n;i++)
            for(int j=0;j<n;j++)
                if(a[i][j]<min && a[i][j]!=0)
                {
                    min=a[i][j];
                    x=i;
                    y=j;
                }
    }
```

```

int u=find(x);
int v=find(y);

if(u!=v)
{
    printf("%d-%d=%d\n",x,y,min);
    cost+=min;
    uni(u,v);
    edges++;
}

a[x][y]=a[y][x]=999;
}

printf("Cost=%d",cost);
}

```

Output:

```

08b_KruskalsAlgo.c > main()
28
29     int edges=0;

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab" && gcc 08b_KruskalsAlgo.c -o 08b_KruskalsAlgo && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\08b_KruskalsAlgo
2-3=4
0-3=5
0-2=6
Cost=15
c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>

```

Lab program 9: Fractional Knapsack

```
#include <stdio.h>

int main()
{
    int n,W;

    printf("Enter items and capacity: ");
    scanf("%d%d",&n,&W);

    float wt[n],profit[n],ratio[n];

    for(int i=0;i<n;i++)
        scanf("%f%f",&wt[i],&profit[i]);

    for(int i=0;i<n;i++)
        ratio[i]=profit[i]/wt[i];

    for(int i=0;i<n-1;i++)
        for(int j=i+1;j<n;j++)
            if(ratio[i]<ratio[j])
            {
                float t;

                t=ratio[i]; ratio[i]=ratio[j]; ratio[j]=t;
                t=wt[i]; wt[i]=wt[j]; wt[j]=t;
                t=profit[i]; profit[i]=profit[j]; profit[j]=t;
            }

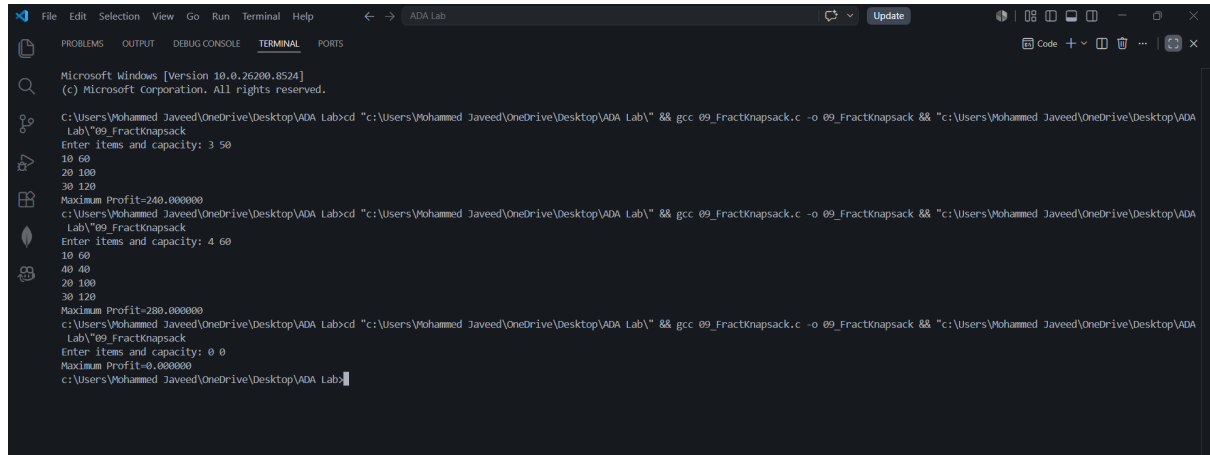
    float total=0;

    for(int i=0;i<n;i++)
    {
        if(W>=wt[i])
        {
            total+=profit[i];
            W-=wt[i];
        }
        else
        {
            total+=ratio[i]*W;
            break;
        }
    }
}
```



```
printf("Maximum Profit=%f",total);  
}
```

Output:



```
Microsoft Windows [Version 10.0.26700.8524]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 09_FractionalKnapsack.c -o 09_FractionalKnapsack && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\09_FractionalKnapsack  
Enter items and capacity: 3 50  
10 60  
20 100  
30 120  
Maximum Profit=240.000000  
C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 09_FractionalKnapsack.c -o 09_FractionalKnapsack && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\09_FractionalKnapsack  
Enter items and capacity: 4 60  
10 60  
40 40  
20 100  
30 120  
Maximum Profit=280.000000  
C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 09_FractionalKnapsack.c -o 09_FractionalKnapsack && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\09_FractionalKnapsack  
Enter items and capacity: 0 0  
Maximum Profit=0.000000  
C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>
```

Lab program 10: Dijkstra's Algorithm

```
#include <stdio.h>

#define INF 9999

int minDist(int dist[], int vis[], int n)
{
    int min = INF, pos;

    for(int i = 0; i < n; i++)
    {
        if(!vis[i] && dist[i] < min)
        {
            min = dist[i];
            pos = i;
        }
    }

    return pos;
}

int main()
{
    int n, src;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int g[n][n], dist[n], vis[n];

    printf("Enter adjacency matrix:\n");

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            scanf("%d", &g[i][j]);

    printf("Enter source vertex: ");
    scanf("%d", &src);

    for(int i = 0; i < n; i++)
    {
        dist[i] = INF;
        vis[i] = 0;
    }
}
```

```

dist[src] = 0;

for(int i = 0; i < n - 1; i++)
{
    int u = minDist(dist, vis, n);

    vis[u] = 1;

    for(int v = 0; v < n; v++)
    {
        if(!vis[v] && g[u][v] &&
            dist[u] + g[u][v] < dist[v])
        {
            dist[v] = dist[u] + g[u][v];
        }
    }
}

printf("Shortest distances:\n");

for(int i = 0; i < n; i++)
    printf("%d -> %d = %d\n", src, i, dist[i]);

return 0;
}

```

Output:

```

Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 10_Dijkstra.c -o 10_Dijkstra && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\"10_Dijkstra
Enter number of vertices: 5
Enter adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter source vertex: 0
Shortest distances:
0 -> 0 = 0
0 -> 1 = 10
0 -> 2 = 50
0 -> 3 = 30
0 -> 4 = 60

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 10_Dijkstra.c -o 10_Dijkstra && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\"10_Dijkstra
Enter number of vertices: 4
Enter adjacency matrix:
0 5 2 0
5 0 1 3
2 1 0 4
0 3 4 0
Enter source vertex: 0
Shortest distances:
0 -> 0 = 0
0 -> 1 = 3
0 -> 2 = 2
0 -> 3 = 6

c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>

```

Lab program 11: N-Queens

```
#include <stdio.h>

int board[20], n;

int safe(int row, int col)
{
    for(int i = 0; i < row; i++)
    {
        if(board[i] == col ||
           board[i] - i == col - row ||
           board[i] + i == col + row)
            return 0;
    }

    return 1;
}

void solve(int row)
{
    if(row == n)
    {
        for(int i = 0; i < n; i++)
            printf("%d ", board[i]);

        printf("\n");
        return;
    }

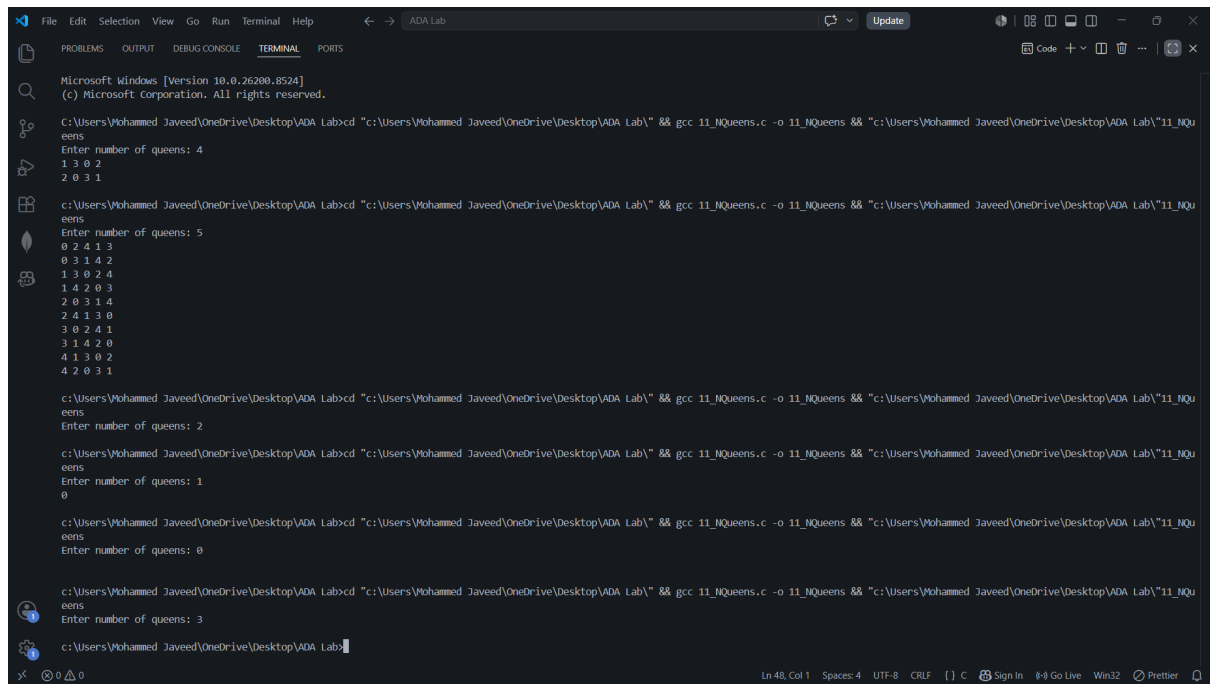
    for(int col = 0; col < n; col++)
    {
        if(safe(row, col))
        {
            board[row] = col;
            solve(row + 1);
        }
    }
}

int main()
{
    printf("Enter number of queens: ");
    scanf("%d", &n);

    solve(0);
}
```

```
    return 0;
}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 11_NQueens.c -o 11_NQueens && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\11_NQueens.exe"
Enter number of queens: 4
1 3 0 2
2 0 3 1

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 11_NQueens.c -o 11_NQueens && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\11_NQueens.exe"
Enter number of queens: 5
0 2 4 1 3
0 3 1 4 2
1 3 0 2 4
1 4 2 0 3
2 0 3 1 4
2 4 1 3 0
3 0 2 4 1
3 1 4 2 0
4 1 3 0 2
4 2 0 3 1

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 11_NQueens.c -o 11_NQueens && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\11_NQueens.exe"
Enter number of queens: 2

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 11_NQueens.c -o 11_NQueens && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\11_NQueens.exe"
Enter number of queens: 1
0

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 11_NQueens.c -o 11_NQueens && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\11_NQueens.exe"
Enter number of queens: 0

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>cd "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\" && gcc 11_NQueens.c -o 11_NQueens && "c:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab\11_NQueens.exe"
Enter number of queens: 3

C:\Users\Mohammed Javeed\OneDrive\Desktop\ADA Lab>
```